Reviewer Pack — Minimal Hodge-Structure Dimension and Circuit Monotone Width…

Entry fb975f2c1541 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 14:48:11 UTC

Minimal Hodge-Structure Dimension and Circuit Monotone Width Inequality

Entry ID: `fb975f2c1541` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 14:48:11 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every satisfiable Boolean circuit C with n inputs, the minimal dimension of a Hodge-structure that can represent the algebraic variety associated with C is at least Ω(log n), and this bound holds with constant multiplicative factors.

Rationale (proposer's reasoning):

> Hodge theory provides a rich algebraic structure for studying varieties. The conjecture suggests that the complexity of Boolean circuits, measured by their monotone width, can be related to the algebraic geometry of these varieties. If true, it would link algebraic geometry to circuit complexity in a new way, potentially revealing hidden structures in both fields.

Taxonomy category: `circuit_monotone_width` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `e59ba83bd4c24af0`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all generated satisfiable Boolean circuits C with n inputs ($n \leq 40$), the ratio of the minimal Hodge-structure dimension to log n is at least Ω(1) and this holds true for all seeds. The criterion is falsified if any seed produces a ratio less than 0.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge theory" AND "circuit monotone width" - "algebraic geometry" AND "Boolean circuit complexity" AND minimal dimension - "minimal Hodge-structure dimension" AND Circuit Monotone Width Inequality

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
from fractions import Fraction
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        circuit = []
        for _ in range(2**n):
            truth_table = [random.randint(0, 1) for _ in range(n)]
            output = random.randint(0, 1)
            circuit.append((truth_table, output))
        return circuit

    def monotone_width(circuit):
        n = len(circuit[0][0])
        max_width = 0
        for i in range(n):
            width = sum(1 for truth_table, _ in circuit if truth_table[i] == 1)
            max_width = max(max_width, width)
        return max_width

    def hodge_dimension(truth_table):
        n = len(truth_table)
        # Simplified Hodge dimension calculation (not actual Hodge theory)
        return sum(1 for bit in truth_table if bit == 1)

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        circuit = generate_circuit(n)
        width = monotone_width(circuit)
        hodge_dim = sum(hodge_dimension(truth_table) for truth_table, _ in circuit)
```

```
        ratio = Fraction(hodge_dim, n).limit_denominator()

        results.append({
            "n": n,
            "width": width,
            "hodge_dim": hodge_dim,
            "ratio": ratio
        })

    metric_value = sum(result["ratio"] for result in results) / len(results)
    conjecture_holds = all(result["ratio"] >= Fraction(1, 2) for result in results)
    counterexample = "" if conjecture_holds else f"Ratio: {min(result['ratio'] for result in resul

    return {
        "metric_name": "Hodge-Structure Dimension to Log(n) Ratio",
        "metric_value": float(metric_value),
        "instances_tested": len(results),
        "n_max": max(result["n"] for result in results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"Ratio too low\" first_failing_seed={first_faili
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means it did not complete within the allotted time limit. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14841
2	propose	ollama_remote	glm4:latest	0	0	19911
3	propose	ollama_remote	glm4:latest	0	0	13527
4	preregistration	ollama_remote	glm4:latest	0	0	9609
5	novelty	ollama_remote	glm4:latest	0	0	10447
6	novelty	ollama_remote	glm4:latest	0	0	13456
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16326
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8496
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10838
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8986
11	judge	ollama_remote	glm4:latest	0	0	32329

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 158766 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/fb975f2c1541.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fb975f2c1541.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/fb975f2c1541.tar.gz` (if generated)